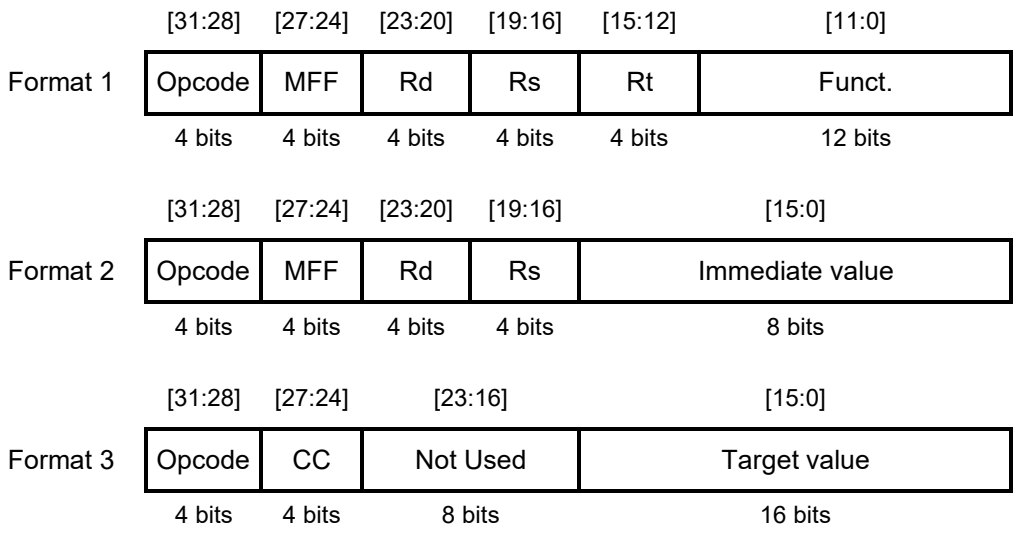
SISC Instruction Set

Overview:



Instruction	*Operation*	*Format*	*Opcode*	*MFF*	*Funct*
NOP	None.	1	0000 *(0x0)*	0000 *(0x0)*	0 *(0x000)*
ADD	Rd ← Rs + Rt	1	0001 *(0x1)*	0001 *(0x1)*	N/U
SUB	Rd ← Rs – Rt	1	0001 *(0x1)*	0010 *(0x2)*	N/U
ADC	Rd ← Rs + Rt + C	1	0001 *(0x1)*	0011 *(0x3)*	N/U
NOT	Rd ← ~Rs	1	0001 *(0x1)*	0100 *(0x4)*	N/U
OR	Rd ← Rs ∨ Rt	1	0001 *(0x1)*	0101 *(0x5)*	N/U
AND	Rd ← Rs ∧ Rt	1	0001 *(0x1)*	0110 *(0x6)*	N/U
XOR	Rd ← Rs ⊕ Rt	1	0001 *(0x1)*	0111 *(0x7)*	N/U
ROR	Rd ← Rs ⩾ [Rt[4:0]]	1	0001 *(0x1)*	1000 *(0x8)*	N/U
ROL	Rd ← Rs ⩽ [Rt[4:0]]	1	0001 *(0x1)*	1001 *(0x9)*	N/U
SHR	Rd ← Rs >> [Rt[4:0]]	1	0001 *(0x1)*	1010 *(0xA)*	N/U
SHL	Rd ← Rs << [Rt[4:0]]	1	0001 *(0x1)*	1011 *(0xB)*	N/U
RRC	C,Rd ← C,Rs ⩾ [Rt[4:0]]	1	0001 *(0x1)*	1100 *(0xC)*	N/U
RLC	C,Rd ← Rs,C ⩽ [Rt[4:0]]	1	0001 *(0x1)*	1101 *(0xD)*	N/U
ASR	C,Rd ← Rs >>> [Rt[4:0]]	1	0001 *(0x1)*	1110 *(0xE)*	N/U
ASL	C,Rd ← Rs <<< [Rt[4:0]]	1	0001 *(0x1)*	1111 *(0xF)*	N/U
ADI	Rd ← Rs + (se) imm	2	0010 *(0x2)*	0001 *(0x1)*	N/A
SBI	Rd ← Rs – (se) imm	2	0010 *(0x2)*	0002 *(0x2)*	N/A
ACI	Rd ← Rs + (se) imm + C	2	0010 *(0x2)*	0003 *(0x3)*	N/A
NTI	Rd ← ~(zf) imm	2	0002 *(0x1)*	0100 *(0x4)*	N/A

Instruction	*Operation*	*Format*	*Opcode*	*MFF*	*Funct*
ORI	Rd ← Rs ∨ (zf) imm	2	0010 *(0x2)*	0101 *(0x5)*	N/A
ANI	Rd ← Rs ∧ (zf) imm	2	0010 *(0x2)*	0110 *(0x6)*	N/A
XRI	Rd ← Rs ⊕ (zf) imm	2	0010 *(0x2)*	0111 *(0x7)*	N/A
RORI	Rd ← Rs ≥ imm[4:0]	2	0010 *(0x2)*	1000 *(0x8)*	N/A
ROLI	Rd ← Rs ≤ imm[4:0]	2	0010 *(0x2)*	1001 *(0x9)*	N/A
SHRI	Rd ← Rs >> imm[4:0]	2	0010 *(0x2)*	1010 *(0xA)*	N/A
SHLI	Rd ← Rs << imm[4:0]	2	0010 *(0x2)*	1011 *(0xB)*	N/A
RRCI	C,Rd ← C,Rs ≥ imm[4:0]	2	0010 *(0x2)*	1100 *(0xC)*	N/A
RLCI	C,Rd ← Rs,C ≤ imm[4:0]	2	0010 *(0x2)*	1101 *(0xD)*	N/A
ASRI	C,Rd ← Rs >>> imm[4:0]	2	0010 *(0x2)*	1110 *(0xE)*	N/A
ASLI	C,Rd ← Rs <<< imm[4:0]	2	0010 *(0x2)*	1111 *(0xF)*	N/A
SWP	Rs ← Rd, Rd ← Rs	2	0011 *(0x3)*	N/A	N/A
BRA	PC ← target (if taken)	3	0100 *(0x4)*	CC	N/A
BRR	PC ← (PC+1) + target (if taken)	3	0101 *(0x5)*	CC	N/A
BNE	PC ← target (if taken)	3	0110 *(0x6)*	CC	N/A
BNR	PC ← (PC+1) + target (if taken)	3	0111 *(0x7)*	CC	N/A
JPA	PC ← target	3	1000 *(0x8)*	CC	N/A
JPR	PC ← (PC+1) + target	3	1001 *(0x9)*	CC	N/A
LDA	Rd ← Mem[imm]	2	1010 *(0xA)*	0000 *(0x0)*	N/A
LDX	Rd ← Mem[Rs + imm]	2	1010 *(0xA)*	1000 *(0x8)*	N/A
LDR	Rd ← Mem[Rs] Rs ← Rs + imm	2	1010 *(0xA)*	0001 *(0x1)*	N/A
LDP	Rd ← Mem[Rs + imm] Rs ← Rs + imm	2	1010 *(0xA)*	1001 *(0x9)*	N/A
STA	Mem[imm] ← Rd	2	1011 *(0xB)*	0000 *(0x0)*	N/A
STX	Mem[Rs + imm] ← Rd	2	1011 *(0xB)*	1000 *(0x8)*	N/A
STR	Mem[Rs] ← Rd Rs ← Rs + imm	2	1011 *(0xB)*	0001 *(0x1)*	N/A
STP	Mem[Rs + imm] ← Rd Rs ← Rs + imm	2	1011 *(0xB)*	1001 *(0x9)*	N/A
CLA	PC ← immediate Rd ← PC + 1	2	1100 *(0xC)*	0000 *(0x0)*	N/A
CLR	PC ← (PC+1) + immediate Rd ← PC + 1	2	1100 *(0xC)*	1000 *(0x8)*	N/A
RET	PC ← Rd	2	1101 *(0xD)*	0000 *(0x0)*	N/A
HLT	Stop Execution	3	1111 *(0xF)*	N/A	N/A

Details/Notes:

CC:

Upon executing arithmetic, logical or shift instructions, the ALU outputs a four-bit status code that describes the result of the operation, which is saved in the status register (statreg.v) for use in branch instructions. The four bits are referred to as C, V, N, and Z:

- Bit 3 (Carry): This bit of the status register is set high when the operation generates a carry or borrow.
- Bit 2 (oVerflow): This bit is set high when the addition or subtraction causes an overflow or underflow.
- Bit 1 (Negative): This bit is set high when the result of the addition or subtraction *should* be negative. Note that if the addition of two positive numbers overflows, this bit will **not** be set high, because the result *should* be positive. Similarly, if the addition of two negative numbers underflows, it **will** be set high, though the ALU output will be a positive number.
- Bit 0 (Zero): This bit is set high when the addition or subtraction results in zero.

Note that only the status bits that are relevant to the selected operation will be updated. For arithmetic instructions, all four status bits are updated. For logical instructions, only the N and Z bits are updated. For shift and rotate instructions, only the N and Z bits are updated, except for the ASR, ASL, RRC, and RLC, where the C bit is also updated.

BRA, BRR, BNE, and BNR:

For the branch instructions, the CC field of the instruction in combination with the current status register contents determine if the branch is taken or not. If the branch is not taken, the PC is incremented to point to the next instruction following the branch instruction. If the branch is taken, the PC is loaded with the branch target address.

In the case of logical-positive branches (BRA & BRR), the branch is taken when *any* of the instruction's CC bits are set to 1, AND the corresponding bits in the status register are also 1 (i.e., (CC & STAT) != 0).

To illustrate this, consider the case when we wish to add two registers, then branch if the result of that addition is less than or equal to zero. We would use a BRA or BRR instruction with the CC = 0011, because we want to take the branch if *either* the negative status bit or the zero status bit is set in the status register.

Similarly, for logical-negative branches (BNE & BNR), the branch is *not* taken if the status register bit is set high for any CC bit set to 1 (i.e., (CC & STAT) == 0).

If, in the above example, we wished instead to take the branch whenever the addition was greater or equal to zero, we would use a BNE or BNR instruction with the CC = 0010. That way, the only case in which we will *not* take the branch is when the ALU sets the negative status bit, and we will therefore branch for all ALU outputs that are zero or greater.

Lastly, it should be noted that a BNE or BNR instruction with the CC = 0000 is effectively an unconditional branch.

IMM:

In the ADI, ACI, and SBI instructions, the 16-bit immediate value is sign-extended to create a 32-bit operand to be added to/subtracted from Rs. If, for some reason, you need an immediate value that does not fit in 16 bits, use a temporary register to hold the top 16 bits, shift them left 16 places, and add them to the lower 16 bits.

For the logical immediate instructions, the 16-bit immediate value is zero-filled for the upper 16 bits before performing the logical operation. As above, a 32-bit immediate value can be constructed by shifting and ORing values.

For immediate shift/rotate operations, only the lower 5 bits of the immediate value is used as the shift count.

Because the PC is only 16 bits long, the immediate values do not need to be sign-extended for the branch, load, and store instructions. Addresses in both instruction memory and data memory are 16 bits long, so immediate values can span all of the addressable space.

LDP, LDR, STP, and STR Instructions:

For the LDP and STP instructions, if the IMM value is +1/-1, the effective addressing mode is +/-(Rs), i.e., pre-increment or pre-decrement. For the LDR and STR instructions, if the IMM value is +1/-1, the effective addressing mode is (Rs)+/-, i.e., post-increment or post-decrement.

Pseudo Instructions:

Many common functions are accomplished by the use of specific opcodes, registers, and immediate values. Some of them are as follows.

Pseudo Instruction	Real Instruction	Comments
CLR Rd	ADD Rd,R0,R0	Clear a register
LRI Rd,imm	ADI Rd,R0,imm	Load a register with sign extended immediate value
INC Rd	ADI Rd,Rd,0x0001	Increment a register
DEC Rd	ADI Rd,Rd,0xFFFF	Decrement a register
STI Rd,Rs	STX Rd,Rs,0x0000	Store indirect; M[Rs] ← Rd
LDI Rd,Rs	LDX Rd,Rs,0x0000	Load indirect; Rd ← M[Rs]
NEG Rd	SUB Rd,R0,Rd	Negate Rd; i.e. Rd ← -Rd